# E-COMMERCE PROFIT PREDICTION

# Complete Python Code Explanation

Study guide • Code walkthrough • Machine Learning concepts • Viva preparation

**Algorithms:** Linear Regression and Random Forest Regressor

**Dataset:** Cleaned_master_Olist_Data.csv

Prepared from the 605-line Python script shared in this conversation.

## Project workflow

The goal is to estimate the profit column using product and purchase-context features. The script loads and cleans the Olist dataset, engineers time features, splits the data chronologically, preprocesses inputs, trains two regression models, compares their metrics, saves the selected pipeline and predicts an illustrative new order.

| Step | Explanation |
|---|---|
| 1. Load | Read the CSV into a Pandas DataFrame. |
| 2. Validate and clean | Check required columns, deduplicate item rows and convert data types. |
| 3. Feature engineering | Extract purchase month, weekday and hour. |
| 4. Prepare data | Set profit as y and selected columns as X. |
| 5. Split | Earlier 80% for training; later 20% for testing. |
| 6. Preprocess | Impute missing values, scale numeric columns and encode categories. |
| 7. Train | Fit Linear Regression and Random Forest. |
| 8. Evaluate | Calculate MAE, RMSE and $R^2$; select lowest-MAE model. |
| 9. Save and predict | Save the fitted pipeline and estimate a new example order. |

## 1. Imports (Lines 1–26)

```
import os
import warnings
import joblib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import (
    mean_absolute_error, mean_squared_error, r2_score
)
warnings.filterwarnings("ignore")
```

**os** — Checks file and folder paths.

**warnings** — Controls warning messages.

**joblib** — Saves and loads the trained model pipeline.

**NumPy** — Performs numerical calculations such as square roots and finite-value checks.

**Pandas** — Reads and manipulates tabular data.

**Matplotlib** — Creates graphs.

**ColumnTransformer** — Applies different preprocessing to different column groups.

**Pipeline** — Chains preprocessing and model steps.

**OneHotEncoder** — Converts categories into numerical indicator columns.

**StandardScaler** — Standardizes numerical features.

**SimpleImputer** — Fills missing values.

**LinearRegression / RandomForestRegressor** — The two regression algorithms.

**MAE / MSE / R² functions** — Measure model prediction performance.

**warnings.filterwarnings('ignore')** — Hides warnings; it does not fix errors.

## 2. Load dataset (Lines 28–51)

```
FILE_PATH = "Cleaned_master_Olist_Data.csv"
if not os.path.exists(FILE_PATH):
```

```
        raise FileNotFoundError(
            f"Dataset not found: {FILE_PATH}\n"
            "Place the CSV file in the same folder as this script."
        )
df = pd.read_csv(FILE_PATH)
print("Dataset shape:", df.shape)
print(df.columns.tolist())
print(df.head())
```

**FILE_PATH** — Stores the CSV filename in one variable.

**os.path.exists** — Checks whether the file exists.

**if not** — Runs the error block if the file is missing.

**raise FileNotFoundError** — Stops execution with a clear message.

**pd.read_csv** — Loads the CSV into DataFrame df.

**df.shape** — Returns number of rows and columns.

**df.columns.tolist()** — Displays column names as a Python list.

**df.head()** — Displays the first five rows.

## 3. Check required columns (Lines 53–75)

```
required_columns = [
    "order_id", "order_item_id", "product_id", "seller_id",
    "order_purchase_timestamp", "profit", "price"
]
missing_columns = [
    col for col in required_columns if col not in df.columns
]
if missing_columns:
    raise ValueError(
        f"Required columns missing: {missing_columns}"
    )
```

**required_columns** — Lists the columns needed by the script.

**List comprehension** — Checks each required name and collects names absent from df.

**if missing_columns** — Checks whether the missing-column list is non-empty.

**raise ValueError** — Stops the program and reports missing columns.

## 4. Remove duplicate item rows (Lines 77–101)

```
item_key = ["order_id", "order_item_id", "product_id", "seller_id"]
before = len(df)
df = df.drop_duplicates(
    subset=item_key, keep="first"
).copy()
print("Rows before:", before)
print("Rows after:", len(df))
```

**item_key** — The four columns together are treated as the order-item identifier.

**len(df)** — Counts rows before deduplication.

**drop_duplicates(subset=...)** — Removes rows with repeated values across all four key columns.

**keep='first'** — Keeps the first row from each duplicate group.

**copy()** — Creates a separate copy of the retained data.

**Rows after** — Shows the remaining row count.

**Data-quality caution:** Verify that duplicate rows do not contain conflicting profit, price, freight or payment values. Keeping the first row is safe only if it is representative.

## 5. Convert data types and clean target (Lines 103–144)

```
df["order_purchase_timestamp"] = pd.to_datetime(
    df["order_purchase_timestamp"], errors="coerce"
)
numeric_columns = [
    "profit", "price", "product_weight_g",
    "product_length_cm", "product_height_cm",
    "product_width_cm", "product_name_lenght",
```

```
    "product_description_lenght", "product_photos_qty"
]
for col in numeric_columns:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors="coerce")
df = df.dropna(
    subset=["profit", "order_purchase_timestamp"]
).copy()
df = df[np.isfinite(df["profit"])].copy()
```

**pd.to_datetime** — Converts timestamp strings into datetime values.

**errors='coerce'** — Invalid dates become NaT.

**numeric_columns** — Lists columns expected to contain numbers.

**for col ...** — Loops through the numeric column names.

**if col in df.columns** — Only converts columns that actually exist.

**pd.to_numeric** — Converts values to numbers; invalid values become NaN.

**dropna(subset=...)** — Removes rows missing profit or purchase timestamp.

**np.isfinite** — Excludes infinite or non-finite profit values.

## 6. Feature engineering (Lines 145–160)

```
df["purchase_month"] = (
    df["order_purchase_timestamp"].dt.month
)
df["purchase_day_of_week"] = (
    df["order_purchase_timestamp"].dt.dayofweek
)
df["purchase_hour"] = (
    df["order_purchase_timestamp"].dt.hour
)
```

**.dt.month** — Extracts month number from 1 to 12.

**.dt.dayofweek** — Extracts weekday: Monday=0 through Sunday=6.

**.dt.hour** — Extracts hour from 0 to 23.

**Feature engineering** — Creates useful model inputs from an existing timestamp.

## 7. Define target and features (Lines 163–237)

```
TARGET = "profit"
numeric_features = [
    "price", "product_weight_g", "product_length_cm",
    "product_height_cm", "product_width_cm",
    "product_name_lenght", "product_description_lenght",
    "product_photos_qty", "purchase_month",
    "purchase_day_of_week", "purchase_hour"
]
categorical_features = [
    "product_category_name_english", "seller_state", "customer_state"
]
numeric_features = [
    col for col in numeric_features if col in df.columns
]
categorical_features = [
    col for col in categorical_features if col in df.columns
]
feature_columns = numeric_features + categorical_features
df = df.dropna(subset=feature_columns, how="all").copy()
X = df[feature_columns].copy()
y = df[TARGET].copy()
for col in categorical_features:
    X[col] = X[col].apply(
        lambda value: str(value) if pd.notna(value) else np.nan
    )
```

**TARGET** — Sets profit as the target variable y.

**numeric_features** — Lists numeric inputs such as price, dimensions and time fields.

**categorical_features** — Lists category and state labels.

**if col in df.columns** — Filters out optional features not present in the CSV.

**feature_columns** — Combines numeric and categorical feature names.

**dropna(how='all')** — Removes rows only when all selected input features are missing.

**X** — DataFrame containing the model inputs.

**y** — Series containing actual profit values.

**lambda / pd.notna / str** — Converts non-missing category values to strings and preserves missing values.

**Target leakage caution:** Product cost, freight, discount and return loss are excluded. If profit is calculated directly from these components, including them could let the model reconstruct the target formula.

## 8. Chronological train-test split (Lines 239–290)

```
df = df.sort_values(
    "order_purchase_timestamp"
).reset_index(drop=True)
X = df[feature_columns].copy()
y = df[TARGET].copy()
split_index = int(len(df) * 0.80)
X_train = X.iloc[:split_index].copy()
X_test = X.iloc[split_index:].copy()
y_train = y.iloc[:split_index].copy()
y_test = y.iloc[split_index:].copy()
```

**sort_values** — Sorts records from oldest purchase to newest.

**reset_index(drop=True)** — Resets row numbering after sorting.

**Recreate X and y** — Keeps input and target rows aligned with sorted data.

**int(len(df) * 0.80)** — Calculates the 80% boundary.

**iloc[:split_index]** — Selects the first 80% for training.

**iloc[split_index:]** — Selects the final 20% for testing.

**X_train/y_train** — Inputs and targets used to train the models.

**X_test/y_test** — Unseen inputs and actual targets used for evaluation.

**Chronology caution:** This is a row-position split after sorting. If many records share timestamps, consider a date-based cutoff and inspect the boundary.

## 9. Preprocessing (Lines 292–343)

```
numeric_transformer = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler())
    ]
)
categorical_transformer = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("encoder", OneHotEncoder(handle_unknown="ignore"))
    ]
)
preprocessor = ColumnTransformer(
    transformers=[
        ("numeric", numeric_transformer, numeric_features),
        ("categorical", categorical_transformer, categorical_features)
    ],
    remainder="drop"
)
```

**Numeric median imputer** — Fills missing numeric values with the median learned from training data.

**StandardScaler** — Standardizes numeric features using training mean and standard deviation.

**Most-frequent imputer** — Fills missing categories with the most frequent training category.

**OneHotEncoder** — Creates indicator columns for categories.

**handle_unknown='ignore'** — Allows categories not seen during training without crashing.

**ColumnTransformer** — Applies the numeric and categorical pipelines to their respective columns.

**remainder='drop'** — Drops columns not explicitly listed.

## 10. Define ML models (Lines 346–379)

```
linear_model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("model", LinearRegression())
    ]
)
random_forest_model = Pipeline(
    steps=[
        ("preprocessor", preprocessor),
        ("model", RandomForestRegressor(
            n_estimators=200, max_depth=20,
            min_samples_split=5, min_samples_leaf=2,
            random_state=42, n_jobs=-1
        ))
    ]
)
```

**LinearRegression()** — Fits a linear relationship between input features and profit.

**RandomForestRegressor** — Builds multiple decision trees and averages their regression predictions.

**n_estimators=200** — Builds 200 trees.

**max_depth=20** — Limits each tree to depth 20.

**min_samples_split=5** — Requires at least five samples at a node before splitting.

**min_samples_leaf=2** — Requires at least two samples in a leaf.

**random_state=42** — Makes random choices reproducible.

**n_jobs=-1** — Uses all available CPU cores where supported.

**Pipeline** — Preprocesses inputs before passing them to the estimator.

## 11. Train and evaluate (Lines 381–437)

```
models = {
    "Linear Regression": linear_model,
    "Random Forest": random_forest_model
}
results = {}
predictions = {}
for model_name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    mae = mean_absolute_error(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    r2 = r2_score(y_test, y_pred)
    results[model_name] = {
        "MAE": mae, "RMSE": rmse, "R2": r2
    }
    predictions[model_name] = y_pred
```

**models dictionary** — Stores both estimators under readable names.

**results / predictions** — Empty dictionaries for metrics and predicted values.

**models.items()** — Loops through each model name and model object.

**model.fit(X_train, y_train)** — Fits preprocessing and trains the estimator on training data.

**model.predict(X_test)** — Predicts profit for unseen test inputs.

**MAE** — Mean absolute error: average absolute difference; lower is better.

**RMSE** — Root mean squared error; larger errors have greater influence; lower is better.

**$R^2$** — Compares model performance with predicting the test-set mean; higher is generally better and it can be negative.

**results[model_name]** — Stores the metrics for the current model.

**predictions[model_name]** — Stores test predictions for later use.

## 12. Compare models (Lines 438–461)

```
results_df = pd.DataFrame(results).T
print(results_df.round(4))
results_df.to_csv("model_comparison.csv")
```

```
best_model_name = results_df["MAE"].idxmin()
```

**pd.DataFrame(results)** — Converts the metrics dictionary into a DataFrame.

**.T** — Transposes the table so models are rows and metrics are columns.

**round(4)** — Rounds displayed values to four decimal places.

**to_csv** — Saves the model comparison table.

**results_df['MAE'].idxmin()** — Returns the model name with the smallest MAE.

**Selection rule:** The script chooses by MAE only. The model with lowest MAE may not also have the lowest RMSE or highest $R^2$; report all metrics.

## 13. Actual vs predicted table (Lines 463–487)

```
best_model = models[best_model_name]
best_predictions = predictions[best_model_name]
comparison_df = pd.DataFrame({
    "Actual_Profit": y_test.to_numpy(),
    "Predicted_Profit": best_predictions
})
comparison_df["Prediction_Error"] = (
    comparison_df["Actual_Profit"]
    - comparison_df["Predicted_Profit"]
)
comparison_df.to_csv("actual_vs_predicted.csv", index=False)
```

**best_model** — Retrieves the selected fitted pipeline.

**best_predictions** — Retrieves the selected model's test predictions.

**DataFrame** — Pairs actual and predicted profit values row by row.

**to_numpy()** — Converts the Pandas Series to a NumPy array.

**Prediction_Error** — Calculates actual minus predicted profit.

**Positive error** — Means underprediction: predicted profit was lower than actual.

**Negative error** — Means overprediction: predicted profit was higher than actual.

**index=False** — Exports the CSV without the DataFrame index.

## 14. Scatter plot (Lines 489–533)

```
plt.figure(figsize=(9, 6))
plt.scatter(
    comparison_df["Actual_Profit"],
    comparison_df["Predicted_Profit"],
    alpha=0.5
)
min_value = min(
    comparison_df["Actual_Profit"].min(),
    comparison_df["Predicted_Profit"].min()
)
max_value = max(
    comparison_df["Actual_Profit"].max(),
    comparison_df["Predicted_Profit"].max()
)
plt.plot(
    [min_value, max_value],
    [min_value, max_value],
    linestyle="--"
)
plt.xlabel("Actual Profit")
plt.ylabel("Predicted Profit")
plt.title(f"Actual vs Predicted Profit ({best_model_name})")
plt.tight_layout()
plt.savefig("actual_vs_predicted.png", dpi=300)
plt.show()
```

**figure(figsize=(9, 6))** — Creates a 9-by-6-inch plotting area.

**scatter(actual, predicted)** — Plots actual profit on X and predicted profit on Y; each dot is a test record.

**alpha=0.5** — Makes points semi-transparent.

**min_value / max_value** — Find the common minimum and maximum across actual and predicted values.

**plt.plot(..., linestyle='--')** — Draws the y=x reference line where predictions are exact.

**Points above the line** — Predicted profit is higher than actual profit: overprediction.

**Points below the line** — Predicted profit is lower than actual profit: underprediction.

**xlabel / ylabel / title** — Set axis labels and a title with the chosen model name.

**tight_layout()** — Adjusts spacing around plot elements.

**savefig(..., dpi=300)** — Saves a high-resolution PNG.

**show()** — Displays the graph.

## 15. Save trained model (Lines 535–546)

```
MODEL_PATH = "profit_prediction_model.pkl"
joblib.dump(best_model, MODEL_PATH)
print("Model saved successfully:", MODEL_PATH)
```

**MODEL_PATH** — Stores the model filename.

**joblib.dump** — Saves the fitted preprocessing-and-model pipeline to a .pkl file.

**Why save?** — The saved pipeline can be reused without training again.

## 16. Predict a new order (Lines 548–602)

```
new_order = {
    "price": 150.0,
    "product_weight_g": 1200.0,
    "product_length_cm": 30.0,
    "product_height_cm": 15.0,
    "product_width_cm": 20.0,
    "product_name_lenght": 40.0,
    "product_description_lenght": 300.0,
    "product_photos_qty": 3.0,
    "purchase_month": 10,
    "purchase_day_of_week": 2,
    "purchase_hour": 14,
    "product_category_name_english": "housewares",
    "seller_state": "SP",
    "customer_state": "SP"
}
new_order_df = pd.DataFrame([new_order]).reindex(
    columns=feature_columns
)
loaded_model = joblib.load(MODEL_PATH)
predicted_profit = loaded_model.predict(new_order_df)[0]
print(f"Estimated profit: {predicted_profit:.2f}")
```

**new_order** — Dictionary containing illustrative values for each input feature.

**DataFrame([new_order])** — Converts the dictionary into a one-row DataFrame.

**reindex(columns=feature_columns)** — Matches the model's expected columns and order; absent columns become NaN.

**joblib.load** — Loads the saved fitted pipeline.

**predict(new_order_df)** — Applies the fitted preprocessing and model to estimate profit.

**[0]** — Extracts the prediction for the one input row.

**:.2f** — Formats the estimated profit to two decimal places.

**Prediction caution:** The example values are illustrative. The result is an ML estimate, not guaranteed realized profit. The script predicts the existing profit column; it does not calculate accounting profit.

## 17. Output files

| File | Purpose |
|---|---|
| model_comparison.csv | MAE, RMSE and R² for both models. |
| actual_vs_predicted.csv | Actual profit, predicted profit and prediction error for test records. |
| actual_vs_predicted.png | Scatter plot of actual versus predicted profit. |
| profit_prediction_model.pkl | Saved fitted preprocessing and model pipeline. |

## 18. Viva questions and answers

**What is the objective?**
Estimate e-commerce order profit from product and purchase-context features and compare two regression models.

**Why is this regression?**
Profit is a continuous numerical target.

**What are X and y?**
X contains input features; y contains target profit.

**Why Linear Regression?**
It provides a simple baseline and models a linear relationship.

**Why Random Forest?**
It combines decision trees and can learn non-linear patterns.

**What is feature engineering?**
Creating useful inputs from existing data, such as month, weekday and hour.

**What is imputation?**
Filling missing values using a statistic such as median or most frequent value.

**What is One-Hot Encoding?**
Converting categories into binary indicator columns.

**What is StandardScaler?**
Standardizing numeric features using mean and standard deviation.

**Why use a Pipeline?**
It keeps preprocessing and model fitting together and reuses the same transformations.

**Why chronological split?**
It trains on earlier records and evaluates on later records.

**What is MAE?**
Average absolute prediction error.

**What is RMSE?**
Square root of average squared error; it penalizes large errors more.

**What is $R^2$?**
A score comparing model predictions with predicting the test-set mean.

**How is the best model selected?**
The script selects the model with the lowest test MAE.

**Why Joblib?**
To save and reuse the fitted model pipeline.

**What does positive prediction error mean?**
Actual minus predicted is positive, so the model underpredicted.

**Does prediction guarantee profit?**
No. It is an estimate based on historical data and supplied features.

## 19. Checks before final submission

• Verify how the dataset's profit column was calculated. This script uses the existing profit column as y; it does not calculate profit itself.

• Confirm that the four deduplication columns uniquely identify an item and that duplicate rows do not conflict.

• Check units and definitions for price, cost, freight, discount and return-related fields before describing profit.

• Use only features available when a real prediction would be made. Later-known information can cause data leakage.

• Inspect the chronological split boundary, especially if many records share the same timestamp.

• Report MAE, RMSE and $R^2$ together; the code selects the best model using MAE only.

- Feature importance indicates model reliance, not causation, if you add that analysis later.
- Only load trusted .pkl files because serialized Python files can be unsafe from unknown sources.

## One-minute project explanation

My project predicts e-commerce profit using Linear Regression and Random Forest Regressor. First, I load and inspect the cleaned Olist dataset, validate required columns, remove repeated item records and convert data types. I extract month, weekday and hour from the purchase timestamp. I use product and purchase-context information as features and profit as the target. I sort records chronologically and split them into 80% training and 20% testing data. I preprocess numerical features using median imputation and scaling, and categorical features using most-frequent imputation and One-Hot Encoding. Then I train both models and evaluate them using MAE, RMSE and $R^2$. I select the model with the lowest MAE, compare actual and predicted profit, save the fitted pipeline with Joblib and use it to estimate profit for an example new order.